Opalite: geometry-first RGB-D free-space analysis with on-device VLM captioning for assistive navigation

Parker Cai
Khoury College of Computer Sciences
Northeastern University
CS5330 Computer Vision
github.com/ParkerCai/Opalite

***Abstract*—In this project, we built Opalite, a real-time assistive perception system for blind and low-vision users around an Intel RealSense D435i RGB-D camera. The pipeline is local-first. Depth geometry decides direction and distance, a continuous stereo "sonar" turns sector clearance into spatial audio, and an on-demand local visual-language model (Gemma 4 via Ollama) labels what is in front of the user. Geometry handles safety-critical signals. The VLM only adds object names and general scene descriptions. The same C++ core compiles into a Windows desktop GUI, an Android NDK library, and a phone app that drives the D435i over USB-OTG. This report describes the geometry analyzer, the sonification layer, the VLM captioning path, and the Android edge port. It also reports per-frame pipeline latency, depth-alignment validation, and Brain query round-trip time on the development hardware. In practice, this split worked well for the system we built: geometry ran every frame and handled direction and distance, while the VLM answered on demand with object names and short scene descriptions. That kept the geometry path fast and easy to verify, and kept the VLM in a narrow, interactive role.**

I. Problem statement

Blind and low-vision users often rely on a patchwork of tools: one app for OCR, another for scene description, another for navigation, and human help for safety-critical moments. In practice, that is awkward. Navigation is time-sensitive, mobile, noisy, and interactive. The user does not want a long caption. The user wants a short answer like "stairs ahead" or "more open on the left," fast enough to matter.

This project targets that gap. We treat the D435i as a continuously updated depth sensor and the headphones as the output channel. The goal is not full autonomy. It is assistive perception: tell the user where the open path is, sound an alarm when something blocks the forward line, and on request say what the obstacle is.

The need is well documented. The World Health Organization reports that at least 2.2 billion people worldwide live with near or distance vision impairment [1]. The VizWiz benchmark also shows that images captured by blind users create a different problem than standard web-photo datasets: framing is irregular, text can be partial, and the question is usually goal-driven [2]. That framing matters here. We built Opalite for goal-driven assistance, not generic captioning.

II. System overview

Opalite is split into three layered phases that share one C++ core.

Phase 1 is a native Windows desktop GUI that captures aligned color and depth at 848Œ480 / 30 fps from a D435i, deprojects depth into a top-down occupancy view, and scores left / center / right free space from a vertical analysis band over the lower half of the image. We built the GUI with GLFW, Dear ImGui, and OpenGL 3.3, and we used OpenCV for image utilities. Per-frame processing latency is logged to `data/latency.csv` and shown live as median and p95 in the HUD.

Phase 2A (Sonar) turns the per-sector clearance scores into a continuous stereo hum. Closer obstacles raise the amplitude and pulse rate of the corresponding stereo channel. This gives the user directional ambient feedback without needing a screen.

Phase 2B (Brain) is an on-demand caption pane. When the user presses a key, taps the on-screen button on Android, or speaks a question, Opalite JPEG-encodes the current color frame and POSTs it to a local Ollama server running Gemma 4 E2B [3]. The query goes to one of two routes: a *locator* route for "where / find / how far" questions, where Gemma returns `<object>|<side>` and the geometry module builds the spoken sentence using the depth sensor's actual distance at the chosen side, and a *free-form scene* route for everything else, where Gemma writes the full sentence under a no-preamble prompt. In both cases, the geometry analyzer stays the source of truth for metric distance or direction in the reply.

Phase 3 is an Android port. The same C++ modules (`free_space.cpp`, `sonar.cpp`, `brain_client.cpp`) compile into an NDK shared library. A `MainActivity` drives the D435i over USB-OTG via the librealsense Android AAR and exposes a four-pane preview that

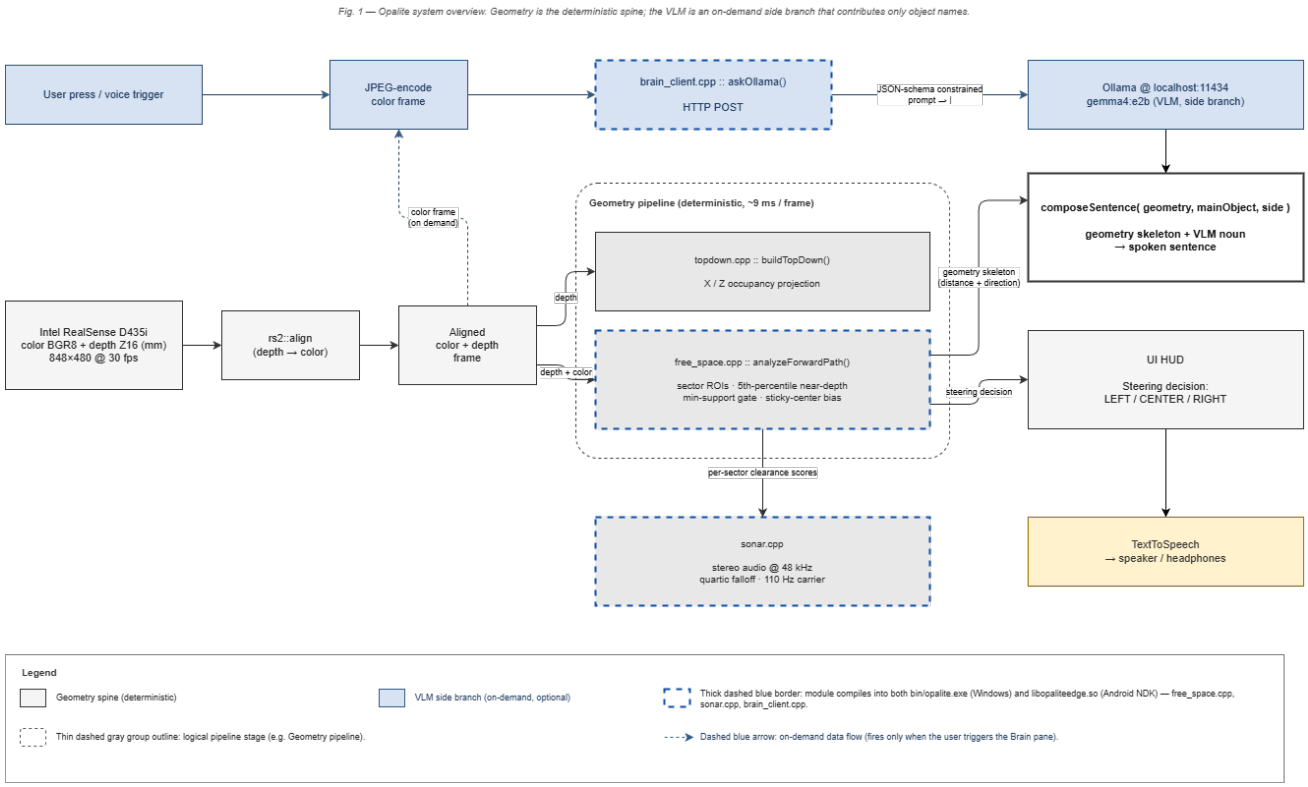

Fig. 1. Opalite system block diagram. The geometry path (depth → free-space analyzer → steering, sonar, sentence skeleton) is the spine. The VLM (Gemma 4 via Ollama) is a side branch that contributes only object names and side labels. The C++ modules in the dashed box compile into both the Windows desktop binary (`bin/opalite.exe`) and the Android NDK shared library (`libopaliteedge.so`).

matches the desktop layout. Output is the phone's headphone jack: spatial sonar plus TextToSpeech for Brain replies. SpeechRecognizer accepts long-press voice commands ("sonar off") or forwards spoken questions to Brain.

III. Computer vision methodology

A. RGB-D capture and alignment

In Phase 1, we use `rs2::pipeline` to stream synchronized color (BGR8) and depth (Z16, millimeters). An `rs2::align` filter resamples depth into the color frame so a pixel index `(u, v)` in the color image maps directly to the same `(u, v)` in the depth image. This invariant lets the sector ROI on the color preview share coordinates with the depth measurements used for steering. We document validation of this alignment in Section V.

The D435i ships with factory-calibrated intrinsics for both the color sensor and the depth (IR) sensor, plus the depth-to-color extrinsic, all retrieved through the librealsense API. The `rs2::align` filter uses these intrinsics and the extrinsic to resample depth into the color image plane, and the top-down projection in Section III-B uses the same intrinsics through `rs2_deproject_pixel_to_point` to convert each pixel `(u, v, Z_mm)` into a metric `(X, Z)` ground point.

B. Free-space geometry analyzer

The core CV module is `analyzeForwardPath()` in `src/free_space.cpp`. It takes the aligned depth frame and a configuration, and returns per-sector near-depth, sector clearance scores in [0, 1], a blocked / clear flag for the center, and a discrete steering decision in {LEFT, CENTER, RIGHT}.

We built the analyzer around five hardening choices that came from live testing:

1. **Vertical analysis band.** We sample only the middle 50% of the depth frame (`coneYFrac = 0.50`). The top quarter (ceiling, distant wall tops) and the bottom quarter (floor and immediate-foreground floor clutter) are excluded so they do not swing the suggested direction.
2. **Non-uniform sectors.** A narrow center beam (~17ř cone, 0.25 of image width) handles forward hazard. Wide left and right sectors share the remaining ~78ř of horizontal field of view for steering.
3. **5th-percentile near-depth, not raw minimum.** For each sector, "how close is the nearest thing" is the 5th percentile of valid depth pixels in the sector, not the absolute minimum. This rejects single dead or speckle pixels that would otherwise mark the sector as blocked.

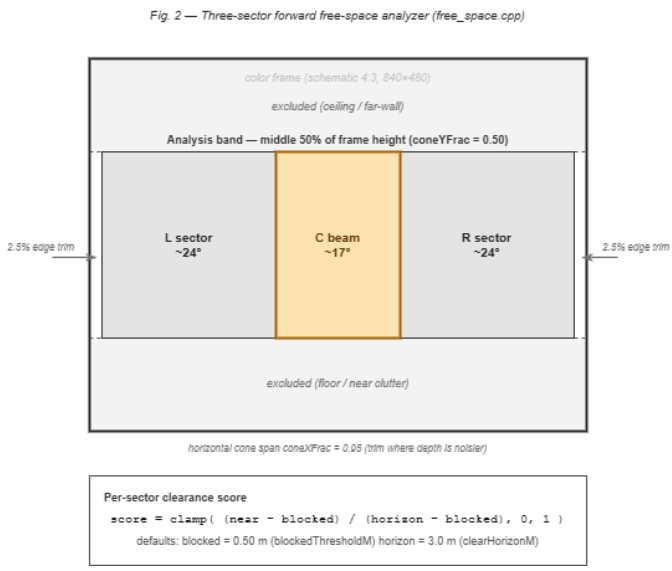

Fig. 2. Three-sector forward free-space analyzer. The middle 50% of the depth frame is sampled. A narrow center beam ($\sim 17°$) handles forward hazard; wide left and right flanks ($\sim 24°$ each) cover steering. Per-sector clearance is scored from the 5th-percentile near-depth, gated by a 200-pixel minimum-support requirement, with a 0.25 m sticky-center bias on the discrete steering decision.

4. **Minimum-support gate.** We trust a sector only when at least 200 valid pixels survive the band crop. Below that, the sector is reported as "insufficient data" and never used to declare a block. This handles tilt, occlusion, and very dark surfaces.
5. **Sticky-center bias.** The discrete steering decision keeps `CENTER` unless a side sector beats center clearance by at least 0.25 m. This prevents the suggested direction from oscillating frame to frame and gives a calmer signal.

Per-sector clearance is computed as

$$\text{score} = \text{clamp}\left(\frac{Z_{\text{near}} - Z_{\text{blocked}}}{Z_{\text{horizon}} - Z_{\text{blocked}}},\ 0,\ 1\right) \tag{1}$$

with a default blocked threshold $Z_{\text{blocked}} = 0.50$ m and a clear horizon $Z_{\text{horizon}} = 3.0$ m.

A separate top-down view (`src/topdown.cpp`) deprojects every depth pixel to a metric `(X, Z)` point and accumulates counts in a 2-D bin grid for visual debugging. It is not on the steering path. It is a visualization aid.

C. Spatial-audio sonification (Phase 2A)

The sonar module (`src/sonar.cpp`) is a miniaudio device running at 48 kHz. The main thread publishes per-sector amplitude targets via `std::atomic` after each geometry frame. The audio callback reads those targets, smooths them sample by sample, and synthesizes a panned 110 Hz carrier with a per-sector envelope. Closer obstacles raise both amplitude and pulse rate. The amplitude curve is `(1 − score)^4` (quartic falloff), so the mid range stays quiet and only real proximity becomes audible. This style of continuous proximity sonification is similar to the auditory display approach in Meijer's vOICe [4], but uses only three coarse spatial sectors rather than per-pixel synthesis. The published amplitudes are also polled at 20 Hz by the UI to render live amplitude meters per sector.

D. VLM-grounded captioning (Phase 2B)

Phase 2B adds on-demand captioning through a local VLM. On a key press or spoken question, Opalite sends the current color frame to Gemma 4 E2B [3] through Ollama and speaks the reply back to the user.

We use two query modes. For position or distance questions such as *where*, *find*, *which side*, and *how far*, a constrained prompt forces the model to return only

`<short_object_description>|<side>`

where *side* is one of *left*, *center*, or *right*. Opalite then combines that label with the most recent depth reading from the chosen side to produce a grounded reply such as *"chair on the left, about one and a half meters."* This keeps the most important spatial information tied to measured geometry instead of asking the VLM to estimate distance on its own.

For more open-ended questions, such as *what's in front of me* or *describe the scene*, the model returns a short scene description. We strip filler phrases from the output so the spoken response starts with useful content. This route is less constrained than the locator route, but it gives the user a more natural scene summary when they want general context rather than a specific location.

We run Gemma with conservative settings so responses stay short and consistent, and we log the round-trip time, route, and success flag for each Brain request in `data/brain_latency.csv`.

E. Android edge port (Phase 3)

Phase 3 ports the same core system to Android. The phone connects to the D435i over USB-OTG, shows a four-pane preview that matches the desktop layout, and plays both the stereo sonar and spoken Brain replies.

This version keeps the same geometry, sonar, and Brain logic from the desktop build, which was an important test of whether the core design would carry over to a mobile device. The app also supports speech input: a long press starts `SpeechRecognizer`, simple commands such as *"sonar off"* are handled directly, and other spoken questions are sent to Brain.

We left OpenCV out of the Android build because the geometry path does not depend on it, and keeping the mobile stack smaller made the port simpler. Audio still runs through miniaudio, while Android `TextToSpeech` handles spoken replies. The point of this port is not the Android-specific code; it is that the geometry analyzer and its parameters generalize to a different output modality (spatial audio + TTS instead of GUI) and a different machine (mobile NDK without OpenCV) without changes to the perception logic.

F. Why this stack is appropriate for an assistive setting

A foundation multimodal model alone is good at general scene understanding, but it hides the intermediate perception steps that matter for safety. In Opalite, we keep those steps explicit. Free-space geometry, sector statistics, support gates, and sticky steering are auditable at the pixel level. The spatial-audio mapping is deterministic. The VLM is reduced to a labeled noun rather than a free-form spatial claim. This separation is what lets the system run fully on the user's device and keeps each component in a role we can justify.

IV. Implementation details

The desktop core lives at the repo root:

- `include/realsense_stream.h`, `src/realsense_stream.cpp`: D435i pipeline + alignment
- `include/free_space.h`, `src/free_space.cpp`: sector ROI + percentile near-depth + steering
- `include/topdown.h`, `src/topdown.cpp`: top-down occupancy projection
- `include/sonar.h`, `src/sonar.cpp`: miniaudio stereo sonar
- `include/brain_client.h`, `src/brain_client.cpp`: Ollama HTTP client
- `include/brain_pane.h`, `src/brain_pane.cpp`: Brain ImGui pane + latency log
- `src/main.cpp`: GLFW + ImGui shell, panel layout, frame loop

Build: CMake 3.24+ with Visual Studio 2022, OpenCV 4.x, and the Intel RealSense SDK 2.0. Third-party pieces (miniaudio 0.11.22, cpp-httplib 0.18.0, nlohmann/json 3.11.3, GLFW 3.3.8, Dear ImGui 1.90.1) are vendored under `third_party/`.

The Android port at `android/opalite-edge/` consumes the same `src/free_space.cpp`, `src/sonar.cpp`, `src/brain_client.cpp` via a separate NDK CMake target plus `app/src/main/cpp/jni_bridge.cpp`. The librealsense AAR is vendored under `app/libs/`. Output is `app-debug.apk` via the standard Gradle build.

Latency telemetry is split into two CSVs: - `data/latency.csv`: per-frame pipeline latency (frame poll success → render draw-list complete), recorded by a 120-frame ring buffer in `LatencyMeter`. - `data/brain_latency.csv`: per-Brain-request round-trip + wall time + mode + success flag.

V. Evaluation setup

The evaluation procedures are documented in `evaluation/TESTING.md` and target two measurable properties of the current system: per-frame pipeline latency and Brain query round-trip time. The earlier scenario-based hazard / object recall harness under `evaluation/results/` was tied to a previous Gemini-Live web prototype and is preserved in the repo as historical context only.

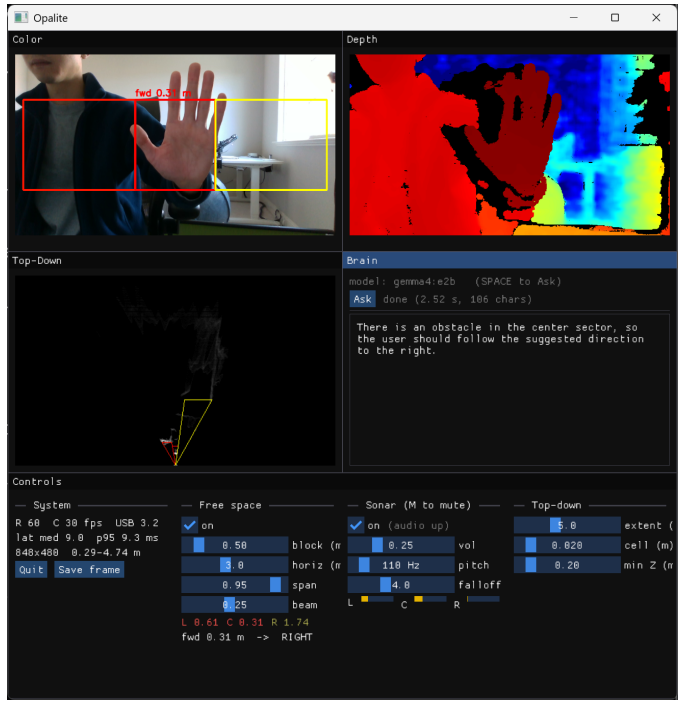

Fig. 3. Live four-pane GUI on the Windows desktop (color, depth, top-down, Brain). A hand obstructs the center beam; the analyzer flags the center sector blocked and the steering decision transitions to RIGHT. The Brain pane caption (free-form route) explicitly references the geometry: "There is an obstacle in the center sector, so the user should follow the suggested direction to the right."

Per-frame pipeline latency. We run the app for at least 30 seconds of normal use, then read `data/latency.csv` offline. The metric is the time from a successful `rs2::pipeline::poll_for_frames()` to the completion of the ImGui draw-list construction for that frame. Sensor exposure, USB transfer, GPU present, and monitor scan-out are not included.

Brain query latency. With Ollama running and Gemma 4 E2B pulled, the user presses Ask ten times with varied scene content, leaving a couple of seconds between presses. The HUD shows the running median over the rolling window, and full round-trip times are written to `data/brain_latency.csv` with a success flag and the route used (locator vs free-form scene).

We check the qualitative behavior of the steering and sonar layers by hand using two scripted scenes: (1) a clear hallway with a single chair moved through the camera's field of view from one side to the other, and (2) a doorway with the user free to step left, center, or right. The expected behavior is documented in `docs/pdr_demo_description_and_script.md` and shown in the demo video.

VI. Results

The results below summarize the current dev-PC measurements and qualitative checks used for this submission.

Phase 1 per-frame pipeline latency (poll → draw-list): median ≈ 9 ms, p95 ≈ 10 ms on the dev PC. The pipeline includes depth colorization, top-down deprojection, sector statistics, top-down rendering, ImGui texture uploads, and draw-list construction. End-to-end sensor-photon-to-screen adds the camera frame period (~33 ms) and GPU present.

Brain query round-trip (Gemma 4 E2B via Ollama, locator route): median ≈ 2.5 s on the dev PC over a ten-press sample with varied scenes. The free-form scene route is slower because the model writes a full sentence. The locator route is faster because the prompt forces a short `<object>|<side>` reply.

Steering behavior (qualitative): in the chair-traversal scene, the suggested direction holds at CENTER while the path is open, transitions to the opposite side once the chair crosses the center beam at less than the blocked threshold (0.50 m), and returns to CENTER once the chair clears the beam. The 0.25 m sticky-center bias prevents per-frame flicker between adjacent sectors when two sides are similarly open. In the doorway scene, the direction tracks the more open side at the door frame and returns to CENTER once the user is through.

Sonar behavior (qualitative): the right-channel hum amplitude rises monotonically as the chair approaches from the right, with the quartic falloff keeping the mid range quiet so the foreground change is the audible event rather than the background.

VII. Discussion

The strongest part of this project is the split between geometry and language. The depth analyzer is the only part of the system allowed to declare a blockage or choose a steering direction, so the safety-critical output stays deterministic, auditable, and independent of any cloud service. The VLM is limited to naming or briefly describing what is ahead. If Gemma is wrong, the likely failure is a wrong word, not a wrong direction.

Phase 2A sonar and Phase 3 Android also show that the same geometry logic carries across different outputs and devices. The core analyzer works in the desktop GUI, in continuous stereo audio, and in the phone app without changing the main decision rules. That portability was one of the main design goals.

More broadly, this project shows that a system can still be a strong computer vision project even when it includes a modern VLM. The important part is keeping the perception pipeline explicit and limiting the model to a narrow role. In Opalite, the depth module handles space and distance, while the VLM adds object names and short scene descriptions on demand.

VIII. Limitations

Several limits are obvious.

1. **Active-stereo depth is the main failure mode.** The D435i loses depth on featureless surfaces, in direct sunlight, and on dark / specular materials. The 5th-percentile + min-support gate keeps the sector "insufficient data" rather than wrongly blocked, but the user loses guidance in those scenes.
2. **Single-camera FOV.** A roughly 90ř horizontal FOV cannot see hazards behind or to the sides of the user. Opalite is a forward-path assistant, not a 360ř awareness system.
3. **No formal user study.** All evaluation so far is on the developer plus scripted scenes. Validation with blind or low-vision participants under appropriate supervision is the obvious next step.
4. **Brain depends on a local LAN service in the Android case.** Phase 3 still reaches Gemma 4 over plain HTTP to a host PC running Ollama, not on the phone itself. Fully local multimodal inference on Android is a clear next step, but native runtime support for newer image + text VLMs is still immature, and on current phone hardware it would likely be slower than the host-PC path we use now.
5. **Not a sole mobility aid.** Opalite is an assistive layer. It does not replace a cane, guide dog, or mobility training.

IX. Future work

Concrete next steps:

- collect a recorded campus dataset (paired RGB-D + sonar + Brain logs) for matched offline replay
- run a small pilot with blind or low-vision users to evaluate sonar mapping, sentence brevity, and Brain trust
- add temporal smoothing on the per-sector near-depth so blocked / clear transitions do not flicker on noisy depth
- replace the LAN Brain dependency on Android with fully local multimodal VLM inference once Android runtimes for newer models mature and latency is competitive
- add an explicit OCR module so signs and door numbers can be read without a Brain round-trip
- explore stereo or wider-FOV cameras for a wider steering arc

X. Conclusion

Opalite is a small local-first assistive perception system built around an Intel RealSense D435i. Depth geometry decides direction and distance, a continuous stereo sonar turns sector clearance into spatial audio, and a local Gemma 4 VLM provides nouns on demand. The same C++ core runs on Windows and on an Android phone over USB-OTG. The result is a measurable, auditable computer vision system that gives useful directional and proximity feedback without depending on a cloud multimodal API.

References

[1] World Health Organization. "Vision impairment and blindness." WHO Fact Sheet, 2026. https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment

[2] D. Gurari, Q. Li, A. J. Stangl, A. Guo, C. Lin, K. Grauman, J. Luo, and J. P. Bigham. "VizWiz Grand Challenge: Answering Visual Questions from Blind People." CVPR, 2018. https://openaccess.thecvf.com/content_cvpr_2018/html/Gurari_VizWiz_Grand_Challenge_CVPR_2018_paper.html

[3] Gemma Team, Google DeepMind. "Gemma: Open models based on Gemini research and technology." Technical report, 2024. https://ai.google.dev/gemma

[4] P. B. L. Meijer. "An Experimental System for Auditory Image Representations." IEEE Transactions on Biomedical Engineering, vol. 39, no. 2, pp. 112121, 1992.

[5] L. Keselman, J. I. Woodfill, A. Grunnet-Jepsen, and A. Bhowmik. "Intel RealSense Stereoscopic Depth Cameras." CVPR Workshops, 2017. https://openaccess.thecvf.com/content_cvpr_2017_workshops/w20/html/Keselman_Intel_RealSense_Stereoscopic_CVPR_2017_paper.html

[6] A. Elfes. "Using occupancy grids for mobile robot perception and navigation." Computer, vol. 22, no. 6, pp. 4657, 1989.